



University of Gabès

National Engineering School of Gabès

Department of Electrical Engineering and Automation

Project Report

TecnoBot: Intelligent AI Assistant Based on RAG for Electrical Engineering Education

Supervised by:

Prof. Zaied Mourad

Professor

Prepared by:

Ilyes Harrathi

Malek Chihi

Farah Akkeri

Co-supervised by:

Mme Rania Bengarssalah

Researcher

Academic Year: 2025 – 2026

Contents

1	State of the Art: Artificial Intelligence in Education	8
1.1	Introduction	8
1.2	AI-Driven Educational Systems	9
1.3	Educational Chatbots Based on Large Language Models	10
1.4	Retrieval-Augmented Generation (RAG)	10
1.5	Applications of RAG in Educational Contexts	11
1.6	Conclusion	12
2	Introduction	13
2.1	General Introduction: Artificial Intelligence in Education	13
2.2	AI Pipeline in Educational Systems	14
2.3	Project Introduction: TecnoBot System	15
2.4	Report Organization	16
3	Problem Statement and Proposed Solution	17
3.1	Introduction	17
3.2	Problem Statement	18
3.2.1	Hallucinations in Large Language Models	18
3.2.2	Absence of Domain-Specific Knowledge	19
3.2.3	Data Confidentiality and Privacy Concerns	19
3.2.4	Difficulty of Information Retrieval from Technical PDF Documents	20
3.3	Proposed Solution: TecnoBot	20
3.3.1	Document Ingestion and Knowledge Base Construction	21

3.3.2	Semantic Search and Context Retrieval	21
3.3.3	Local Language Model for Response Generation	22
3.3.4	User Interface and Interaction Layer	22
3.4	Objectives of the Project	23
3.4.1	Develop a Domain-Specific AI Assistant	23
3.4.2	Improve Response Accuracy and Factual Reliability	23
3.4.3	Reduce Hallucinations Through Retrieval-Augmented Generation	23
3.4.4	Enable Semantic Retrieval from Technical PDF Documents	24
3.4.5	Ensure Data Confidentiality Through Local Deployment	24
3.4.6	Support Engineering Students in Their Learning Process	24
3.5	Functional Requirements	24
3.5.1	Document Upload and Ingestion	25
3.5.2	Natural Language Query Processing	25
3.5.3	Semantic Retrieval of Relevant Document Chunks	25
3.5.4	Contextual Response Generation	26
3.5.5	Knowledge Base Management	26
3.5.6	Conversational Interface	26
3.6	Non-Functional Requirements	27
3.6.1	Response Time and Performance	27
3.6.2	Reliability and Availability	28
3.6.3	Scalability	28
3.6.4	Data Privacy and Confidentiality	28
3.6.5	Local Deployment and Infrastructure Independence	29
3.6.6	Maintainability and Extensibility	29
3.6.7	Usability	29
3.7	Conclusion	30
4	System Design and Methodology	32
4.1	Introduction	32
4.2	Phase 1 — Initial Core Architecture with Phi-3	33
4.2.1	General Architecture	33
4.2.2	Streamlit User Interface	34
4.2.2.1	Authentication and Login System	35

4.2.2.2	User Interface — Chatbot Interaction	36
4.2.2.3	Admin Interface — User Management	37
4.2.3	Technical Configuration	38
4.2.3.1	Software Dependencies	38
4.2.3.2	System Setup and Pipeline Initialization	39
4.2.3.3	Initial Test Prompt	39
4.3	Phase 2 — Migration to Qwen 2.5 and Advanced Features	39
4.3.1	Migration from Phi-3 to Qwen 2.5	39
4.3.2	Advanced RAG Pipeline	41
4.3.2.1	Deep Multi-Document Retrieval	42
4.3.2.2	Role-Based Knowledge Access	42
4.3.3	System Dashboard and Architecture Overview	42
4.3.4	Chatbot Interface with Qwen 2.5	43
4.3.5	Interactive Educational Simulations	44
4.3.5.1	AC Signal Simulation	44
4.3.5.2	DC Signal Simulation	45
4.3.5.3	Three-Phase System Simulation	45
4.3.5.4	Ohm’s Law Simulation	46
4.3.5.5	PWM Signal Simulation	47
4.3.6	Advanced User Management	48
4.3.6.1	Role-Based Access Control	48
4.3.7	Conclusion of Phase 2	49
5	Implementation and Experimental Results	50
5.1	Introduction	50
5.2	Development Environment	50
5.2.1	Software Stack	50
5.2.2	Hardware Configuration	51
5.3	System Implementation	52
5.3.1	Document Ingestion Pipeline	52
5.3.2	Embedding Generation and Indexing	52
5.3.3	Retrieval and Generation Pipeline	52
5.4	Experimental Setup	53

5.5	Test Scenarios and Results	53
5.5.1	Scenario 1 — Conceptual Technical Question	54
5.5.2	Scenario 2 — Technical Definition and Formula	54
5.5.3	Scenario 3 — Fault Diagnosis and Troubleshooting	54
5.5.4	Scenario 4 — Electrical Diagnostic Support	54
5.6	Results and Discussion	55
5.6.1	Qualitative Analysis	55
5.6.2	Comparative Analysis: Phi-3 vs Qwen 2.5	55
5.7	Comparison with Traditional LLMs	56
5.8	Limitations	57
5.9	Conclusion	57
5.10	Future Work	60
5.11	Final Remarks	61

List of Figures

2.1	General AI Pipeline in Educational Systems	15
3.1	Overall Architecture of TecnoBot RAG System	21
4.1	Architecture Général de système	33
4.2	Phase 1 Architecture - TecnoBot with Phi-3	34
4.3	TecnoBot Phase 1 — Login Page (Empty State)	35
4.4	TecnoBot Phase 1 — User Login Session	35
4.5	TecnoBot Phase 1 — Administrator Login Session	36
4.6	TecnoBot Phase 1 — User Interface: Diagnostic Support Session	36
4.7	TecnoBot Phase 1 — User Interface: Detailed Diagnostic Response	37
4.8	TecnoBot Phase 1 — Admin Interface: Adding a New User	38
4.9	TecnoBot Phase 1 — Access Control: Admin Section Restricted for Stan- dard Users	38
4.10	TecnoBot Phase 2 — New Authentication Interface (Qwen 2.5 Engine)	41
4.11	Pipeline RAG vertical de TecnoBot	41
4.12	TecnoBot Phase 2 — RAG Pipeline: Live Semantic Search with Source Citations	42
4.13	TecnoBot Phase 2 — System Status Dashboard	43
4.14	TecnoBot Phase 2 — Chatbot Interface: RAG-Grounded Response with Qwen 2.5	44
4.15	TecnoBot Phase 2 — Signal Lab: AC Signal Simulation (407V / 237Hz)	45
4.16	TecnoBot Phase 2 — Signal Lab: DC Signal Simulation (800V, 7% Ripple)	45

4.17 TecnoBot Phase 2 — Signal Lab: Three-Phase System Simulation (400V / 50Hz)	46
4.18 TecnoBot Phase 2 — Signal Lab: Ohm's Law Simulation ($V=230V$, $R=100\Omega$, $f=50Hz$)	47
4.19 TecnoBot Phase 2 — Signal Lab: PWM Signal Simulation (607V, $D=50\%$, $f_{sw}=20kHz$)	47
4.20 TecnoBot Phase 2 — Admin Panel: Registered Users Table	48
4.21 TecnoBot Phase 2 — Admin Panel: New User Creation Form	49
5.1 TecnoBot Future Development Roadmap	61

List of Tables

3.1	Summary of Functional Requirements	27
3.2	Summary of Non-Functional Requirements	30
5.1	TecnoBot Software Stack	51
5.2	Hardware Configuration Used for TecnoBot Deployment	51
5.3	Knowledge Base Dataset Characteristics	53
5.4	Comparative Analysis: Phi-3 vs Qwen 2.5 in TecnoBot	56
5.5	Comparison Between Traditional LLM and TecnoBot RAG Architecture	56

State of the Art: Artificial Intelligence in Education

1.1 Introduction

The rapid evolution of Artificial Intelligence (AI), particularly with the emergence of Large Language Models (LLMs), has significantly transformed the field of education. These advanced models, trained on vast amounts of textual data, are capable of understanding context, generating human-like responses, and assisting users in complex cognitive tasks. As a result, they have enabled the development of intelligent educational systems that support learning, automate academic assistance, and enhance knowledge accessibility [1, 2].

Despite these promising capabilities, traditional AI systems, especially standalone LLMs, present several critical limitations. One of the major challenges lies in their lack of reliability, as these models generate responses based on probabilistic patterns rather than verified knowledge sources. This often leads to inaccuracies, inconsistencies, and contextually irrelevant answers, which can be problematic in educational environments where precision and correctness are essential [3, 4]. Furthermore, LLMs are inherently static once trained, meaning they cannot dynamically update their knowledge without undergoing costly retraining processes.

To overcome these limitations, recent research has increasingly focused on integrating

retrieval mechanisms with generative models, leading to the emergence of Retrieval-Augmented Generation (RAG) systems. This hybrid approach allows AI models to access external, domain-specific knowledge bases in real time, thereby improving the factual accuracy and contextual relevance of generated responses [3]. In this context, this chapter provides a comprehensive review of existing work on AI in education, with a particular focus on LLM-based chatbots and RAG-based educational systems.

1.2 AI-Driven Educational Systems

Artificial Intelligence has been widely adopted in educational systems to enhance learning environments by providing personalized, adaptive, and scalable solutions. Unlike traditional teaching methods, AI-driven platforms are capable of analyzing student interactions, identifying learning gaps, and delivering customized educational content tailored to individual needs. This adaptability plays a crucial role in improving both student engagement and academic performance, particularly in large-scale or online learning environments.

Moreover, the integration of conversational AI has introduced a paradigm shift in the way students interact with learning systems. AI-powered assistants enable natural language interaction, allowing students to ask questions freely and receive immediate, context-aware responses. This significantly reduces psychological barriers, such as fear of judgment, and encourages active participation in the learning process [5]. Additionally, these systems address the long-standing issue of limited individualized attention in classrooms, providing continuous support that complements traditional teaching approaches [5].

In practice, AI technologies have been successfully applied across a wide range of educational domains, including intelligent tutoring systems, automated grading platforms, and language learning applications. Empirical studies have shown that AI-based chatbots can improve student motivation, engagement, and knowledge retention by offering instant feedback and interactive learning experiences [6].

1.3 Educational Chatbots Based on Large Language Models

The emergence of Large Language Models has led to the development of advanced educational chatbots capable of simulating human-like interactions. These systems leverage deep learning techniques to understand complex queries, generate coherent responses, and assist students in various academic tasks, ranging from concept explanation to problem-solving.

Educational chatbots are increasingly used as virtual teaching assistants, providing instant support and facilitating self-directed learning. Their ability to maintain contextual conversations and adapt responses to user inputs makes them particularly effective in modern educational settings.

However, despite these advantages, LLM-based systems suffer from significant limitations. One of the most critical issues is hallucination, where the model generates plausible yet incorrect or fabricated information. This problem undermines the reliability of such systems, especially in technical fields like engineering, where incorrect information can lead to serious consequences [4].

Additionally, LLMs lack access to real-time or domain-specific knowledge, as their responses are constrained by the data used during training. This limitation reduces their effectiveness in specialized educational contexts, where up-to-date and precise information is required [3]. As a result, their direct application in academic environments remains limited without additional mechanisms to ensure accuracy and relevance.

1.4 Retrieval-Augmented Generation (RAG)

To address the shortcomings of traditional LLMs, researchers have proposed Retrieval-Augmented Generation (RAG), a hybrid architecture that combines the strengths of information retrieval and text generation. This approach allows AI systems to access external knowledge sources and use them as a foundation for generating responses.

In a typical RAG pipeline, the system first retrieves relevant documents from a knowledge base using semantic search techniques. These documents are then provided as contextual

input to the language model, enabling it to generate responses grounded in verified information. This significantly improves both the accuracy and reliability of the generated output.

According to Gao et al. (2023), RAG-based systems effectively reduce hallucinations and enhance factual correctness by incorporating external data sources into the generation process [3]. Furthermore, this architecture offers a high degree of flexibility, as the knowledge base can be updated independently of the model, making it well-suited for dynamic and evolving educational content.

1.5 Applications of RAG in Educational Contexts

Recent research has demonstrated the effectiveness of RAG-based systems in various educational applications. These systems are particularly valuable in environments that require accurate, context-aware, and domain-specific information.

Swacha and Gracel (2025) conducted a comprehensive survey of 47 studies on RAG-based educational chatbots, concluding that these systems significantly improve learning outcomes by providing reliable and personalized responses [5]. Their findings highlight the ability of RAG architectures to combine adaptability with high accuracy, making them suitable for modern learning environments.

Similarly, Husain et al. (2024) developed a RAG-based academic chatbot that leverages institutional data to provide verified answers. Their work demonstrates that grounding responses in trusted sources enhances user confidence and system credibility [4].

Experimental studies further confirm these benefits. Thway et al. (2024) reported that integrating a RAG chatbot into a university setting resulted in a 97% student satisfaction rate, along with increased engagement and improved academic performance [7].

In addition, RAG systems have been successfully applied in e-learning platforms, where they enable efficient retrieval of information from educational resources such as PDFs and lecture notes. This capability significantly reduces the time required for information search and enhances the overall learning experience [8].

1.6 Conclusion

The integration of Artificial Intelligence in education has led to significant advancements in intelligent learning systems. In particular, Large Language Model (LLM)-based chatbots have demonstrated strong capabilities in natural language understanding and generation, enabling new forms of interaction between students and educational systems.

However, despite these advantages, LLM-based approaches still present limitations in terms of accuracy, reliability, and contextual grounding. These constraints can reduce their effectiveness in academic and technical domains where precise and verified information is required.

To address these issues, Retrieval-Augmented Generation (RAG) systems have been proposed in recent research as an alternative approach. By combining information retrieval with text generation, RAG architectures improve response accuracy and reduce hallucination risks, while enabling the use of external and updatable knowledge sources.

In this context, the study of RAG-based systems highlights their relevance for educational applications, particularly in domains requiring domain-specific knowledge such as engineering education. This motivates the exploration and implementation of such architectures in intelligent tutoring and assistance systems.

Introduction

2.1 General Introduction: Artificial Intelligence in Education

In recent years, Artificial Intelligence (AI) has emerged as a transformative force in the field of education, reshaping traditional learning methodologies and enabling the development of intelligent, adaptive, and interactive learning environments. AI technologies, particularly those based on Natural Language Processing (NLP), have facilitated the creation of systems capable of understanding human language, analyzing student behavior, and providing personalized educational support [9, 10].

The integration of AI into education has led to the development of intelligent tutoring systems, automated assessment tools, and conversational agents that enhance both teaching and learning processes. These systems are designed to adapt to individual learner profiles, identify knowledge gaps, and deliver customized content, thereby improving learning efficiency and student engagement [11]. Furthermore, AI-powered tools enable continuous feedback and real-time interaction, which are essential components of effective learning environments.

Among recent advancements, generative AI models have introduced new possibilities by allowing machines to generate human-like text, answer complex questions, and assist in problem-solving tasks. These capabilities have made AI systems increasingly relevant in higher education, where students require both conceptual understanding and technical

guidance. However, the deployment of such systems also raises important challenges related to reliability, transparency, and the need for domain-specific knowledge [12].

As a result, current research is increasingly focused on designing AI systems that are not only intelligent but also reliable, interpretable, and adapted to specific educational contexts. This evolution highlights the growing importance of combining advanced AI techniques with structured knowledge sources to support effective learning.

2.2 AI Pipeline in Educational Systems

To better understand the integration of Artificial Intelligence in education, it is useful to describe it as a structured processing pipeline. As shown in Figure 2.1, AI-based educational systems follow a sequence of interconnected stages that transform raw data into meaningful pedagogical interactions.

At the first stage, **data collection** is performed, where educational content such as textbooks, lecture notes, technical manuals, and student interactions are gathered. This data forms the foundation of the system's knowledge base and directly impacts the quality of the generated outputs.

The second stage involves **data processing and representation**. In this phase, textual information is cleaned, segmented, and transformed into numerical representations using embedding techniques. These representations enable the system to capture semantic relationships and support efficient information retrieval.

The third stage corresponds to the **retrieval process**, where the system identifies the most relevant information based on a user query. Instead of relying solely on pre-trained knowledge, modern approaches use semantic search to extract contextually meaningful content from a structured database.

Following retrieval, the **generation stage** is carried out using a language model. The retrieved information is injected as context to generate accurate and coherent responses, ensuring that the output remains aligned with reliable sources.

Finally, the **interaction and feedback stage** allows the system to communicate with the user and potentially refine future responses. This iterative mechanism contributes to

improving both system performance and user experience over time.

This pipeline representation highlights how AI systems in education operate as integrated frameworks combining data processing, retrieval, and generation mechanisms.



Figure 2.1: General AI Pipeline in Educational Systems

2.3 Project Introduction: TecnoBot System

Within this context, the present project proposes the development of an intelligent assistant named **TecnoBot**, designed to support students and technicians in the field of electrical engineering. The main objective of this system is to provide accurate, context-aware, and domain-specific assistance by leveraging recent advances in Artificial Intelligence, particularly Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG).

TecnoBot is conceived as a specialized educational chatbot capable of interacting with users in natural language while relying on a structured database of technical documents. Unlike general-purpose AI systems, which generate responses based on pre-trained knowledge, TecnoBot operates within a controlled environment where all generated answers are grounded in verified sources such as technical manuals, laboratory documents, and academic resources.

The system architecture is based on a RAG pipeline, which combines semantic information retrieval with text generation. When a user submits a query, the system first retrieves the most relevant information from a vectorized database of documents. This retrieved context is then used by a local language model to generate a precise and coherent response. This approach significantly reduces the risk of hallucination and ensures that the generated answers are aligned with the actual content of the knowledge base.

In addition to its core chatbot functionality, TecnoBot is designed as an integrated platform that can support multiple educational features, including technical assistance, concept explanation, and access to structured learning materials. By providing immediate and reliable responses, the system aims to enhance the learning experience, reduce infor-

mation search time, and support students in understanding complex engineering concepts.

The development of TecnoBot also addresses important practical constraints, such as data confidentiality and system accessibility. By operating locally and using institution-specific data, the system ensures that sensitive technical information remains secure while maintaining high availability for users.

2.4 Report Organization

This report is structured into several chapters, each addressing a specific aspect of the project in a logical and progressive manner.

Following this introductory chapter, Chapter 3 presents the problem statement and the proposed solution. It highlights the limitations of existing approaches and explains how the TecnoBot system addresses these challenges through the integration of Retrieval-Augmented Generation (RAG) techniques.

Chapter 4 is dedicated to the methodology adopted in this project. It describes the overall system architecture, the data processing pipeline, and the different design choices made during the development of TecnoBot. This chapter also details the tools, technologies, and frameworks used to implement the solution.

Chapter 5 focuses on the experimental evaluation of the system. It presents the testing procedures, performance analysis, and results obtained under different scenarios. This chapter aims to assess the effectiveness and reliability of the proposed system in real-world conditions.

Finally, Chapter 6 concludes the report by summarizing the main contributions of the project and discussing potential improvements, future research directions, and perspectives for extending the proposed system. This structure ensures a clear and coherent presentation of the problem, the proposed solution, the implementation process, and the obtained results.

Problem Statement and Proposed Solution

3.1 Introduction

The growing demand for accessible and reliable technical knowledge in engineering education has highlighted the limitations of conventional information retrieval methods. Students enrolled in electrical engineering programs are frequently confronted with large volumes of technical documentation, including laboratory manuals, circuit analysis textbooks, and domain-specific standards, making it increasingly difficult to extract precise and contextually relevant information in a timely manner.

Traditional search engines, while effective for general-purpose queries, are not designed to handle the complexity and specificity of technical engineering content. These tools rely on keyword-based indexing mechanisms that often return irrelevant or overly broad results, failing to address the precise informational needs of engineering students [3].

Furthermore, the recent adoption of general-purpose conversational AI systems, such as ChatGPT, has introduced new possibilities for academic assistance. However, these systems are trained on broad, heterogeneous datasets and lack access to institution-specific or domain-restricted knowledge. As a result, their responses in highly specialized technical contexts are often imprecise, unreliable, or disconnected from the actual content of the relevant educational resources [4].

In this context, this chapter presents a structured analysis of the identified limitations, followed by the proposed solution developed within the scope of this project. The TecnoBot

system is introduced as a domain-specific intelligent assistant designed to overcome these challenges by integrating a Retrieval-Augmented Generation (RAG) architecture with a local language model and a structured vector knowledge base.

3.2 Problem Statement

Despite the rapid advancement of Artificial Intelligence in educational contexts, several critical limitations persist in the current approaches, particularly when applied to specialized technical domains such as electrical engineering. These limitations can be categorized into four major challenges, each of which directly impacts the reliability and effectiveness of AI-assisted learning systems.

3.2.1 Hallucinations in Large Language Models

One of the most significant challenges associated with the use of Large Language Models (LLMs) in educational environments is the phenomenon commonly referred to as *hallucination*. This term describes the tendency of generative models to produce responses that are linguistically coherent and contextually plausible, yet factually incorrect or entirely fabricated [13, 14, 3].

In technical domains such as electrical engineering, where precision is paramount, hallucinated responses can have serious consequences. A student relying on an AI system that incorrectly describes the behavior of an electrical circuit, misinterprets a standard formula, or fabricates a non-existent technical specification may develop fundamental misconceptions that are difficult to correct later [15]. This problem is further amplified by the convincing nature of LLM outputs, which often appear authoritative even when they are factually wrong.

The root cause of this issue lies in the architecture of standard LLMs: these models generate responses by predicting statistically likely token sequences based on patterns learned during training, rather than retrieving verified information from trusted sources [13]. As a result, they cannot guarantee factual correctness, especially in highly specialized or rapidly evolving technical fields.

3.2.2 Absence of Domain-Specific Knowledge

General-purpose language models are trained on large, heterogeneous corpora that cover a wide range of subjects. While this broad training enables them to handle diverse queries, it simultaneously limits their depth of knowledge in specialized domains [16]. In the context of electrical engineering education, this limitation is particularly problematic.

Technical content such as laboratory experiment protocols, electrical installation standards, power systems documentation, and control engineering manuals are rarely represented in sufficient detail within the training data of publicly available LLMs. Consequently, when students query these systems about specific topics — such as IEC electrical standards, PLC programming procedures, or transformer protection schemes — the responses generated are often superficial, outdated, or inaccurate [3, 8].

Furthermore, the static nature of pre-trained models means that they cannot incorporate new technical documents or updated standards without undergoing costly and time-consuming retraining processes. This inflexibility renders them unsuitable for dynamic educational environments where knowledge bases must be regularly updated to reflect the latest developments in the field [16].

3.2.3 Data Confidentiality and Privacy Concerns

The deployment of cloud-based AI systems in institutional educational environments raises significant concerns regarding data confidentiality and privacy. When students or faculty members submit queries to external AI platforms, the transmitted data — which may include sensitive institutional documents, unpublished research content, or proprietary technical materials — is processed on remote servers, often outside the jurisdiction of the institution [12, 17].

This represents a critical risk in engineering schools and research institutions, where technical documents, laboratory procedures, and experimental data may be subject to confidentiality agreements or intellectual property regulations. The use of external AI services in such contexts may inadvertently expose sensitive information to third parties, thereby violating institutional privacy policies and potentially compromising ongoing research activities [18].

The absence of locally deployable AI solutions that can operate entirely within a secure institutional environment thus constitutes a major limitation of existing general-purpose AI systems in the context of engineering education.

3.2.4 Difficulty of Information Retrieval from Technical PDF Documents

A significant portion of the technical knowledge relevant to electrical engineering education is stored in PDF format, including textbooks, laboratory manuals, technical datasheets, and normative standards. Despite the widespread use of these documents, retrieving specific and accurate information from large PDF collections remains a challenging task for both students and traditional search tools [8].

Conventional keyword-based search methods are inadequate for navigating the complex and heterogeneous content of technical PDFs. These approaches fail to capture the semantic relationships between concepts, often returning irrelevant results or missing critical information that is expressed using different terminology than the query [19].

Moreover, technical PDF documents frequently contain mixed content types, including mathematical formulas, circuit diagrams, tables, and domain-specific notation, which further complicates automated processing and retrieval [3]. As a result, students are often required to manually browse through extensive documentation to locate the specific information they need, leading to inefficiency and frustration in the learning process.

These four challenges collectively highlight the need for an intelligent, domain-specific, and locally deployable system capable of accurately retrieving and generating reliable technical information. The following section presents the proposed solution developed to address these limitations.

3.3 Proposed Solution: TecnoBot

In response to the limitations identified in the previous section, this project proposes the development of **TecnoBot**, an intelligent domain-specific assistant designed to support students and technicians in the field of electrical engineering. TecnoBot is built upon a *Retrieval-Augmented Generation* (RAG) architecture, which combines the strengths of

semantic information retrieval with the generative capabilities of a local language model, thereby ensuring both accuracy and reliability in the generated responses [16, 3].

The overall architecture of TecnoBot is illustrated in Figure 3.1, and can be decomposed into four main components, each addressing a specific limitation identified in the problem statement.

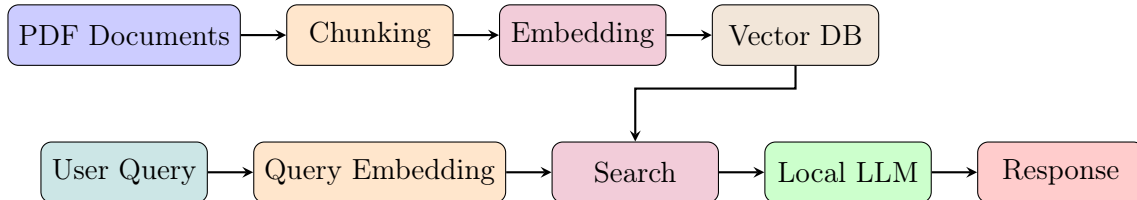


Figure 3.1: Overall Architecture of TecnoBot RAG System

3.3.1 Document Ingestion and Knowledge Base Construction

The first component of TecnoBot is responsible for processing and indexing the technical documents that constitute the system’s knowledge base. In this phase, PDF documents — including laboratory manuals, electrical engineering textbooks, technical standards, and course materials — are ingested, parsed, and segmented into smaller, semantically coherent text chunks.

Each chunk is subsequently transformed into a high-dimensional vector representation using a pre-trained embedding model. These vector representations capture the semantic content of the text, enabling the system to identify conceptually related information even when different terminology is used [19, 20]. The resulting embeddings are stored in a dedicated vector database, which serves as the structured knowledge repository of the system.

3.3.2 Semantic Search and Context Retrieval

When a user submits a query, TecnoBot does not rely on simple keyword matching. Instead, the query is first converted into a vector representation using the same embedding model applied during document indexing. A semantic similarity search is then performed against the vector database to identify the most relevant text chunks with respect to the user’s question [19].

This approach ensures that the retrieved context is semantically aligned with the intent of the query, rather than merely sharing common keywords. By grounding the generation process in verified and contextually relevant document excerpts, the system significantly reduces the risk of hallucination and improves the factual accuracy of the generated responses [3, 16].

3.3.3 Local Language Model for Response Generation

Unlike cloud-based AI systems that rely on external servers for language processing, TecnoBot integrates a **local Large Language Model** that operates entirely within the institutional environment. The retrieved document chunks are injected into the model's context window as supporting evidence, and the language model generates a coherent, precise, and contextually grounded response based on this input [21, 14].

The use of a local LLM addresses the data confidentiality concerns identified in the problem statement, as no sensitive institutional data is transmitted to external platforms. Furthermore, the RAG mechanism ensures that the model's responses remain anchored to the actual content of the knowledge base, preventing the generation of fabricated or misleading information.

3.3.4 User Interface and Interaction Layer

TecnoBot is equipped with an intuitive conversational interface that allows users to interact with the system using natural language. Students and technicians can submit queries, upload new technical documents, and receive structured, accurate responses in real time. The interface is designed to be accessible and easy to use, requiring no prior technical knowledge of AI systems [5].

This integrated architecture positions TecnoBot as a comprehensive solution to the challenges identified in the problem statement, combining domain specificity, factual reliability, data privacy, and ease of use within a single, locally deployable platform.

3.4 Objectives of the Project

The development of TecnoBot is guided by a set of clearly defined objectives that reflect both the technical ambitions and the educational motivations of the project. These objectives are formulated in direct response to the limitations identified in the problem statement and are intended to ensure that the proposed system delivers measurable and meaningful improvements in the context of electrical engineering education.

3.4.1 Develop a Domain-Specific AI Assistant

The primary objective of this project is to design and implement an intelligent assistant that is specifically tailored to the domain of electrical engineering. Unlike general-purpose AI systems, TecnoBot is conceived as a specialized tool that operates exclusively within a controlled knowledge environment composed of verified technical documents. This domain specificity ensures that the system's responses are relevant, precise, and aligned with the actual educational content used in the institution [4, 8].

3.4.2 Improve Response Accuracy and Factual Reliability

A core objective of TecnoBot is to generate responses that are factually correct and verifiable. By grounding the generation process in retrieved document excerpts rather than relying solely on pre-trained model knowledge, the system aims to produce outputs that accurately reflect the content of the knowledge base. This objective directly addresses the reliability limitations of standard LLMs in technical educational contexts [3, 16].

3.4.3 Reduce Hallucinations Through Retrieval-Augmented Generation

One of the most critical objectives of this project is to minimize the occurrence of hallucinations in AI-generated responses. The RAG architecture adopted in TecnoBot ensures that each generated answer is supported by relevant contextual evidence retrieved from verified sources. This retrieval-grounded approach has been shown to significantly reduce hallucination rates compared to standalone LLM systems, thereby increasing the trustworthiness of the assistant [13, 15, 3].

3.4.4 Enable Semantic Retrieval from Technical PDF Documents

TecnoBot aims to provide an efficient and intelligent mechanism for extracting relevant information from large collections of technical PDF documents. Through the use of embedding models and vector-based similarity search, the system enables semantic retrieval that goes beyond simple keyword matching. This capability allows students to obtain precise answers even when their queries are phrased differently from the exact terminology used in the source documents [19, 20].

3.4.5 Ensure Data Confidentiality Through Local Deployment

Given the sensitive nature of institutional technical documents, a key objective of this project is to ensure that all data processing and AI inference are performed locally, without relying on external cloud services. The local deployment of both the language model and the vector database guarantees that no confidential information is transmitted to third-party platforms, thereby preserving institutional data privacy and complying with applicable data protection regulations [12, 18, 17].

3.4.6 Support Engineering Students in Their Learning Process

Beyond its technical objectives, TecnoBot is designed with a strong educational purpose. The system aims to reduce the time students spend searching for technical information, provide immediate and reliable assistance during laboratory sessions and self-study periods, and complement traditional teaching methods by offering an interactive and accessible knowledge resource. By doing so, TecnoBot contributes to improving student engagement, academic performance, and overall learning experience [5, 7, 6].

These objectives collectively define the scope and ambition of the TecnoBot project and serve as the foundation for the design decisions, implementation choices, and evaluation criteria presented in the following chapters.

3.5 Functional Requirements

Functional requirements define the specific capabilities and behaviors that the TecnoBot system must provide in order to fulfill its intended purpose. These requirements are

derived from the project objectives and reflect the core functionalities expected by the end users, namely engineering students and technical staff. Each requirement is formulated to ensure that the system delivers a complete, reliable, and user-centered experience.

3.5.1 Document Upload and Ingestion

The system must provide users with the ability to upload technical documents in PDF format. Upon upload, TecnoBot shall automatically process the submitted files by extracting their textual content, segmenting it into semantically coherent chunks, and indexing the resulting embeddings into the vector database. This functionality enables the continuous enrichment of the knowledge base with new technical resources, including laboratory manuals, course materials, electrical standards, and technical datasheets [8, 3].

The ingestion pipeline must be capable of handling documents of varying lengths and formats, ensuring robust text extraction even from complex PDF structures containing tables, formulas, and mixed layouts.

3.5.2 Natural Language Query Processing

TecnoBot must allow users to interact with the system using natural language, without requiring any knowledge of formal query syntax or programming commands. The system shall accept free-form questions related to electrical engineering topics and process them through the RAG pipeline to generate accurate and contextually relevant responses [16, 4].

The query processing module must support a wide range of question types, including factual questions, conceptual explanations, procedural guidance, and document-specific inquiries. The system should also be capable of handling multi-turn conversations, maintaining contextual coherence across successive user interactions.

3.5.3 Semantic Retrieval of Relevant Document Chunks

Upon receiving a user query, the system must perform a semantic similarity search over the vector database to identify and retrieve the most relevant text chunks from the indexed documents. This retrieval process must be based on vector similarity measures rather than keyword matching, ensuring that conceptually related content is returned even when different terminology is used [19, 20].

The retrieval module shall return a configurable number of top-ranked chunks, ranked according to their semantic relevance to the query. These chunks shall subsequently be passed to the language model as contextual input for response generation.

3.5.4 Contextual Response Generation

The system must generate coherent, accurate, and contextually grounded responses by combining the retrieved document chunks with the user query as input to the local language model. The generated responses shall be directly based on the content of the retrieved chunks, ensuring factual alignment with the knowledge base and minimizing the risk of hallucination [3, 13, 21].

The response generation module must produce outputs that are clear, well-structured, and adapted to the technical level of the target audience. When applicable, the system should indicate the source document from which the relevant information was retrieved, thereby enhancing transparency and user trust.

3.5.5 Knowledge Base Management

TecnoBot must provide basic knowledge base management functionalities, allowing authorized users to add new documents, update existing content, and maintain the integrity of the indexed vector database. This requirement ensures that the system remains up to date with the latest technical resources and educational materials available within the institution [4, 8].

3.5.6 Conversational Interface

The system must offer an intuitive and accessible conversational interface through which users can submit queries, view generated responses, and interact with the system in a natural and fluid manner. The interface shall be designed to minimize friction and ensure ease of use for students with varying levels of technical proficiency [5, 7].

The following table summarizes the functional requirements of the TecnoBot system along with their associated priority levels.

Table 3.1: Summary of Functional Requirements

ID	Functional Requirement	Priority
FR-01	Upload and ingest technical PDF documents into the knowledge base	High
FR-02	Accept and process natural language queries from users	High
FR-03	Perform semantic similarity search over the vector database	High
FR-04	Generate contextually grounded responses using a local LLM	High
FR-05	Display the source document reference for each generated response	Medium
FR-06	Support multi-turn conversational interactions	Medium
FR-07	Allow authorized users to manage and update the knowledge base	Medium
FR-08	Provide an intuitive and accessible conversational interface	High

3.6 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that the TecnoBot system must satisfy in order to ensure a reliable, efficient, and sustainable deployment. Unlike functional requirements, which describe what the system does, non-functional requirements specify how the system performs its functions. These requirements are essential for guaranteeing a high-quality user experience and long-term system viability in an institutional educational environment.

3.6.1 Response Time and Performance

TecnoBot must deliver responses within an acceptable time frame to ensure a smooth and uninterrupted user experience. The system shall be optimized to minimize latency at each stage of the RAG pipeline, including document retrieval, embedding computation, and language model inference. In practice, the total response time for a standard query should not exceed a few seconds under normal operating conditions [7, 8].

Performance optimization must also account for scenarios involving large knowledge bases, where the vector similarity search may become computationally intensive. Efficient indexing strategies and approximate nearest neighbor search algorithms shall be employed to maintain acceptable response times regardless of the size of the document collection

[19].

3.6.2 Reliability and Availability

The system must operate in a stable and consistent manner, providing reliable responses across a wide range of query types and document contents. TecnoBot shall be designed to handle unexpected inputs gracefully, returning informative error messages rather than crashing or producing undefined behavior. The system should maintain high availability during academic periods, ensuring continuous access for students and faculty members [4].

Furthermore, the reliability of the generated responses must be ensured through the RAG mechanism, which grounds each answer in verified document content. This architectural choice inherently contributes to the consistency and trustworthiness of the system's outputs [3, 16].

3.6.3 Scalability

TecnoBot must be designed with scalability in mind, allowing the system to accommodate a growing knowledge base without significant degradation in performance. The vector database architecture shall support the incremental addition of new documents and embeddings, enabling the system to scale horizontally as the volume of technical content increases [8, 19].

In addition, the system architecture should be modular and extensible, allowing future integration of additional language models, embedding strategies, or retrieval mechanisms without requiring a complete redesign of the existing infrastructure [16].

3.6.4 Data Privacy and Confidentiality

As previously emphasized in the problem statement, data privacy constitutes a critical non-functional requirement for TecnoBot. The system must ensure that all data processing operations — including document ingestion, embedding generation, vector storage, and language model inference — are performed entirely within the local institutional environment, without transmitting any information to external servers or third-party cloud platforms [12, 18, 17].

Access to the system and its knowledge base shall be restricted to authorized users, and appropriate security measures must be implemented to prevent unauthorized access to sensitive technical documents. Compliance with applicable data protection regulations and institutional privacy policies is a mandatory requirement for the deployment of TecnoBot.

3.6.5 Local Deployment and Infrastructure Independence

TecnoBot must be fully operational in a local deployment environment, independent of external API services or internet connectivity. This requirement ensures that the system remains functional even in restricted network environments, such as isolated laboratory networks or secure institutional intranets [21, 18].

The system shall be deployable on standard institutional hardware, without requiring specialized or prohibitively expensive computing infrastructure. Compatibility with commonly available server configurations must be ensured during the design and implementation phases.

3.6.6 Maintainability and Extensibility

The codebase and architecture of TecnoBot must be designed according to software engineering best practices, ensuring ease of maintenance, updates, and future extensions. The system shall adopt a modular architecture in which each component — document ingestion, embedding, retrieval, generation, and interface — can be independently modified or replaced without affecting the overall system functionality [4, 5].

Comprehensive documentation of the system architecture, configuration parameters, and deployment procedures shall be provided to facilitate future maintenance and potential handover to other development teams.

3.6.7 Usability

The TecnoBot interface must be intuitive and accessible to users with varying levels of technical expertise. The system shall minimize the learning curve required to interact with the assistant, allowing students to focus on their educational objectives rather than on

system operation. Clear and structured response formatting shall be adopted to enhance readability and comprehension [5, 7, 6].

The following table provides a consolidated summary of the non-functional requirements defined for the TecnoBot system.

Table 3.2: Summary of Non-Functional Requirements

ID	Non-Functional Requirement	Priority
NFR-01	Response time shall not exceed a few seconds under normal conditions	High
NFR-02	System shall operate reliably and handle unexpected inputs gracefully	High
NFR-03	Architecture shall support scalable knowledge base expansion	Medium
NFR-04	All data processing shall be performed locally without external transmission	High
NFR-05	System shall be deployable on standard institutional hardware	High
NFR-06	Codebase shall follow modular and maintainable design principles	Medium
NFR-07	Interface shall be intuitive and accessible to non-technical users	High
NFR-08	Access to the knowledge base shall be restricted to authorized users	High

3.7 Conclusion

This chapter has presented a comprehensive analysis of the challenges associated with the use of conventional AI systems in technical educational environments, with a particular focus on electrical engineering education. The identified limitations — including hallucinations in large language models, the absence of domain-specific knowledge, data confidentiality concerns, and the difficulty of retrieving information from technical PDF documents — collectively highlight the need for a more reliable, specialized, and privacy-preserving intelligent assistant.

In response to these challenges, the TecnoBot system has been proposed as a domain-specific solution based on a Retrieval-Augmented Generation (RAG) architecture. By combining semantic information retrieval with local language model inference, TecnoBot

addresses the core limitations of standalone LLM systems while ensuring that all generated responses are grounded in verified and institution-specific technical knowledge [3, 16, 4].

The project objectives, functional requirements, and non-functional requirements defined throughout this chapter provide a clear and structured foundation for the design and implementation of the system. These specifications ensure that TecnoBot is developed as a reliable, scalable, and user-centered platform that meets the practical needs of engineering students and technical staff.

The following chapter will present the methodology and system architecture adopted for the implementation of TecnoBot, detailing the technical choices made at each stage of the RAG pipeline, from document ingestion and embedding to semantic retrieval and response generation.

System Design and Methodology

4.1 Introduction

This chapter presents the complete design methodology and system architecture adopted throughout the development of TecnoBot. The project was conducted following an iterative and progressive approach, structured into two major development phases, each introducing significant improvements in terms of system performance, user interface quality, and educational functionalities.

The first phase established the foundational architecture of the system, integrating the Phi-3 language model with a basic RAG pipeline and a Streamlit-based user interface. The second phase extended this foundation by migrating to the more capable Qwen 2.5 model and introducing advanced features including interactive electrical simulations, an enriched admin dashboard, and a deep retrieval mechanism.

Throughout both phases, the core design principle remained consistent: all AI inference and document retrieval operations are performed locally, ensuring data confidentiality and institutional independence from external cloud services [21, 16]. The RAG architecture serves as the backbone of the system, combining semantic document retrieval with language model generation to produce reliable, context-grounded responses for engineering students and technical staff.

The following sections describe in detail the architecture, interface design, and technical configuration adopted at each phase of the project.

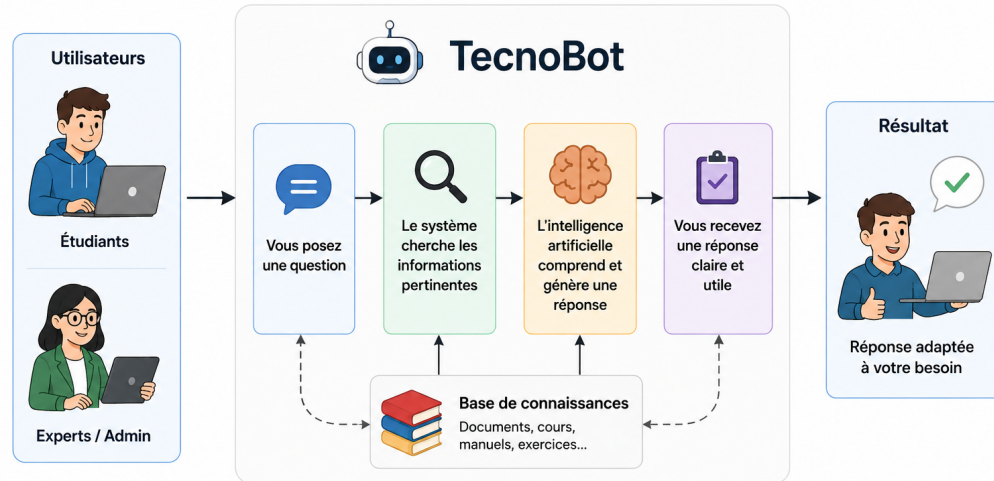


Figure 4.1: Architecture Général de système

4.2 Phase 1 — Initial Core Architecture with Phi-3

4.2.1 General Architecture

The first phase of TecnoBot’s development focused on establishing a functional and operational RAG-based system capable of answering technical questions in the domain of electrical engineering. The core architecture of this phase is built around the **Phi-3** language model, a compact yet capable local LLM developed by Microsoft, chosen for its efficiency on standard hardware and its ability to generate coherent technical responses [22].

The overall system architecture of Phase 1 is structured around five main components, as illustrated in Figure 4.2:

1. **PDF Loader:** Technical documents are loaded from a local directory using a PDF parsing module. These documents include laboratory manuals, electrical engineering course materials, and technical standards relevant to the target domain.
2. **Text Chunking and Embedding:** The extracted text is segmented into overlapping chunks using a recursive character-based splitter. Each chunk is subsequently

transformed into a vector representation using a pre-trained sentence embedding model [20].

3. **ChromaDB Vector Store:** The generated embeddings are stored in a persistent ChromaDB vector database, which serves as the structured knowledge repository of the system. ChromaDB enables efficient semantic similarity search over large document collections [3].
4. **Retriever and RAG Pipeline:** Upon receiving a user query, the system retrieves the most semantically relevant chunks from ChromaDB and injects them as contextual evidence into the language model prompt, following the standard RAG architecture [16].
5. **Phi-3 LLM (via Ollama):** The augmented prompt is processed by the Phi-3 model running locally through the Ollama inference framework, which generates a coherent and contextually grounded response.

The system is further divided into two distinct access layers: a **User Interface** allowing students to submit technical queries and receive AI-generated responses, and an **Admin Interface** enabling authorized personnel to manage users and upload new documents into the knowledge base.

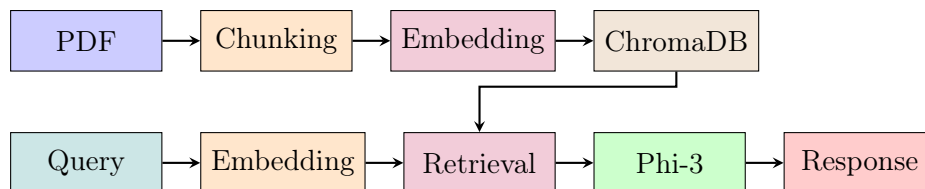


Figure 4.2: Phase 1 Architecture - TecnoBot with Phi-3

4.2.2 Streamlit User Interface

The user-facing component of TecnoBot Phase 1 was developed using **Streamlit**, a Python-based framework for building interactive web applications. The interface was designed to be minimal, intuitive, and accessible to users without prior technical knowledge of AI systems.

4.2.2.1 Authentication and Login System

The system implements a role-based authentication mechanism that controls access to the different components of the platform. Upon launching the application, users are presented with a login page requiring a username and password. Based on the provided credentials, the system redirects the authenticated user to either the standard user interface or the administrative panel.

Figures 4.3, 4.4, and 4.5 illustrate the login interface of TecnoBot Phase 1, showing the authentication page in its default state, a standard user login session, and an administrator login session respectively.

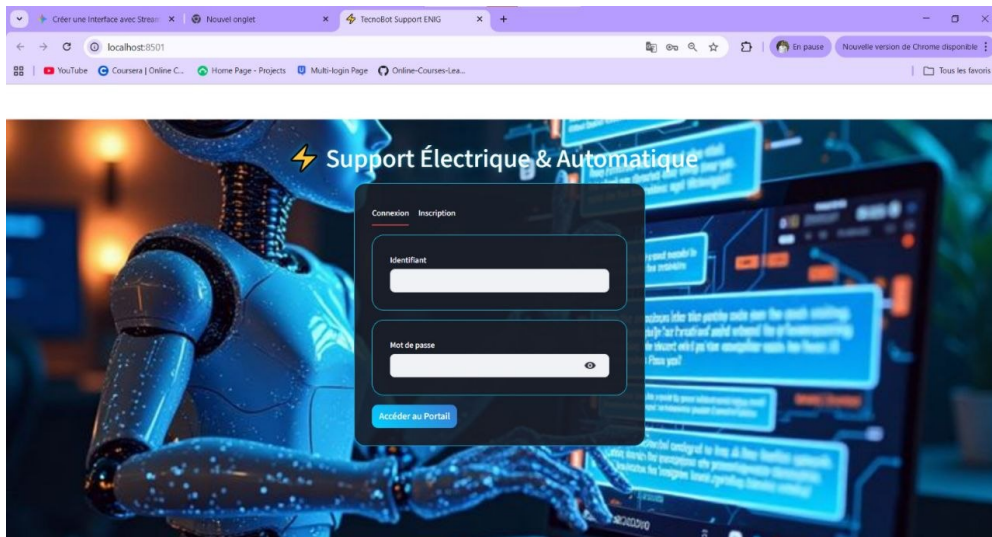


Figure 4.3: TecnoBot Phase 1 — Login Page (Empty State)

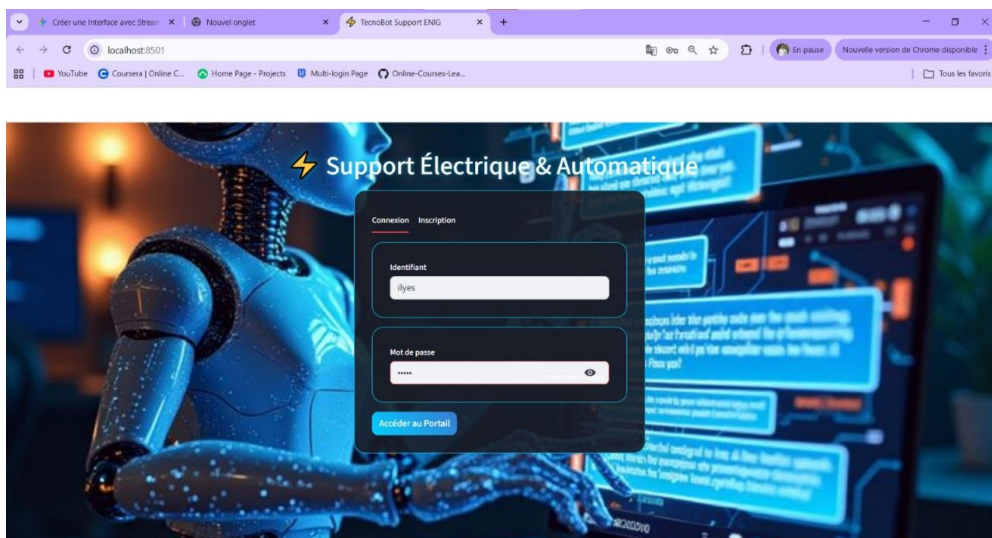


Figure 4.4: TecnoBot Phase 1 — User Login Session

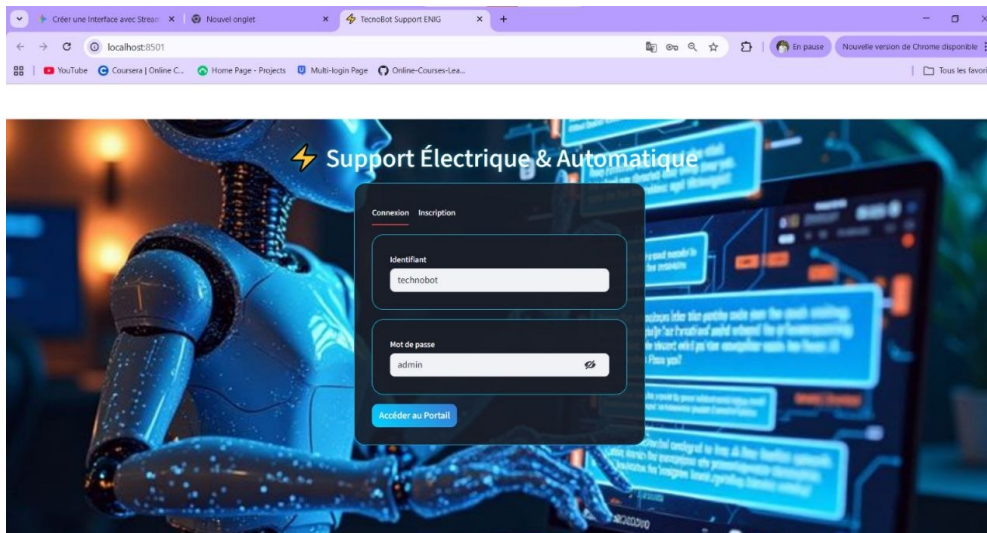


Figure 4.5: TecnoBot Phase 1 — Administrator Login Session

4.2.2.2 User Interface — Chatbot Interaction

Once authenticated as a standard user, the student is redirected to the main chatbot interface. This interface provides a natural language input field through which the user can submit technical queries related to electrical engineering topics. The system processes the query through the RAG pipeline and displays the generated response directly in the conversation area.

Figure 4.6 illustrates the main user interface of TecnoBot Phase 1, showing a diagnostic support session in which the system successfully identified a potential electrical fault.

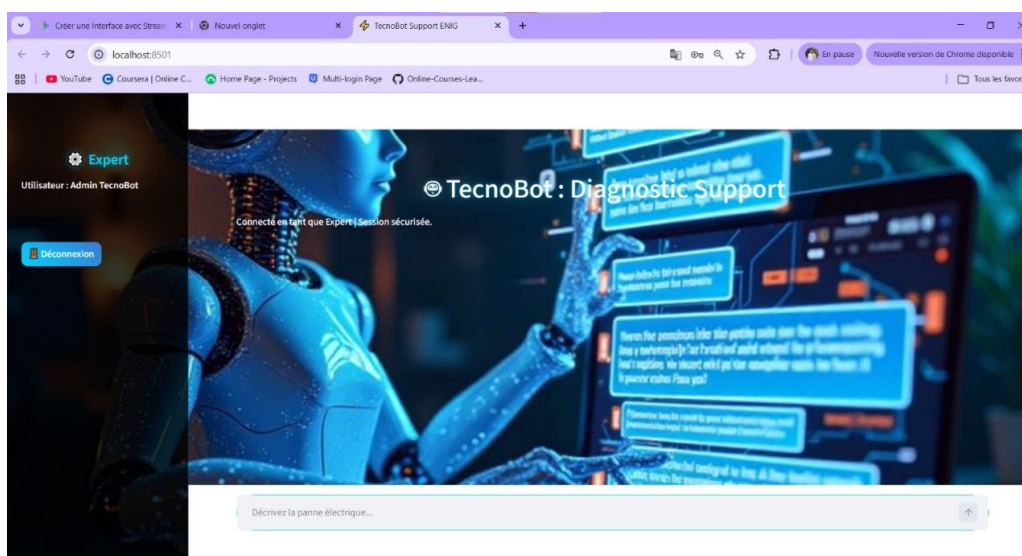


Figure 4.6: TecnoBot Phase 1 — User Interface: Diagnostic Support Session

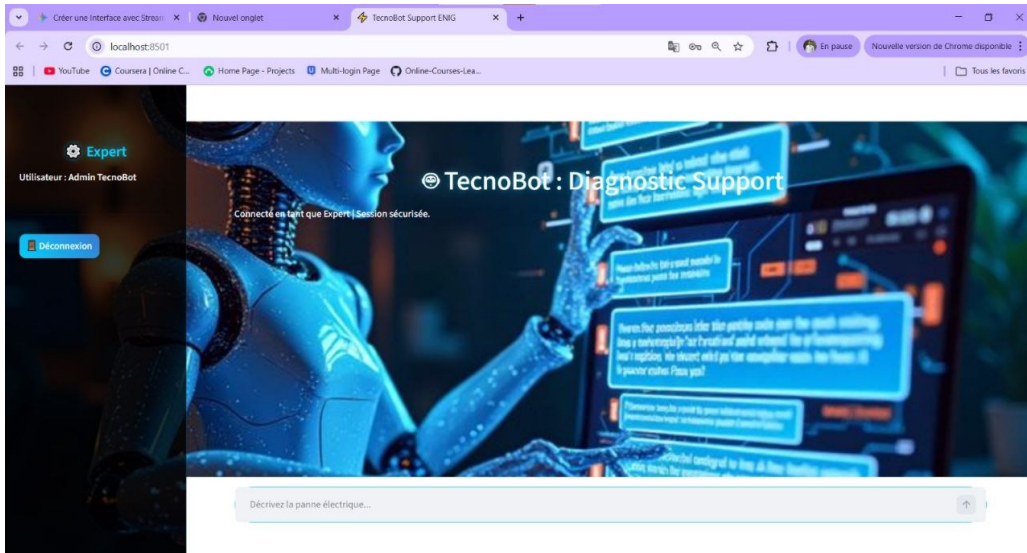


Figure 4.7: TecnoBot Phase 1 — User Interface: Detailed Diagnostic Response

4.2.2.3 Admin Interface — User Management

The administrative interface provides authorized users with a dedicated panel for managing the platform's user base and knowledge base. Key functionalities of the admin interface in Phase 1 include:

- Adding new users with assigned roles
- Viewing the list of registered users
- Deleting existing user accounts
- Uploading new PDF documents into the knowledge base

Figure 4.8 illustrates the user addition functionality of the admin panel, where a new member named *rania* with the role *Étudiant* has been successfully created. Figure 4.9 shows the registered users list displaying the existing accounts along with their assigned roles and creation timestamps.

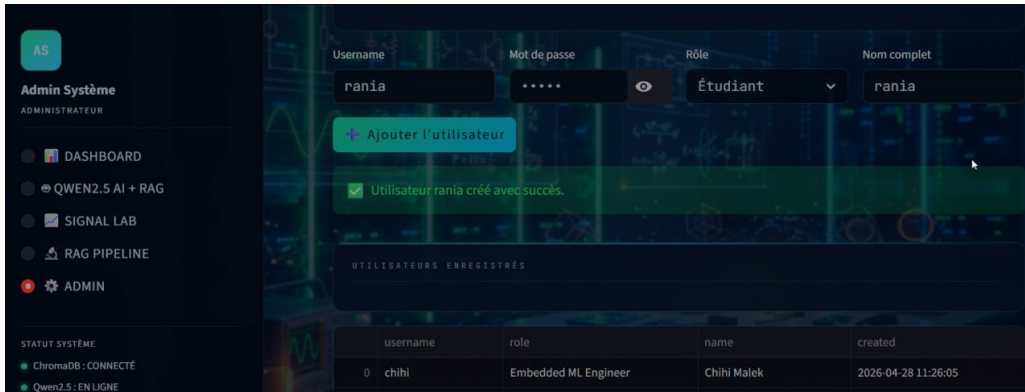


Figure 4.8: TecnoBot Phase 1 — Admin Interface: Adding a New User

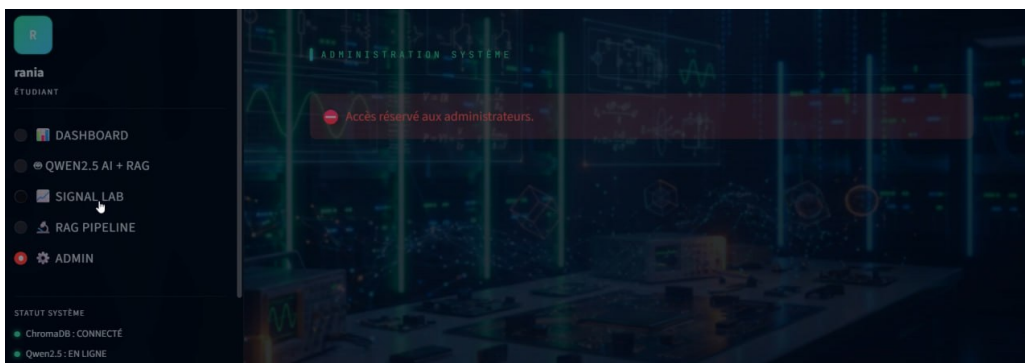


Figure 4.9: TecnoBot Phase 1 — Access Control: Admin Section Restricted for Standard Users

4.2.3 Technical Configuration

The deployment of TecnoBot Phase 1 requires the installation and configuration of several software components. The following steps describe the complete setup procedure adopted during the initial development phase.

4.2.3.1 Software Dependencies

The system is built on a Python-based stack. The required dependencies include:

- **Python 3.10+:** Core programming language
- **Ollama:** Local LLM inference framework for running Phi-3
- **Streamlit:** Web interface framework
- **ChromaDB:** Persistent vector database for embedding storage
- **LangChain:** Orchestration framework for the RAG pipeline

- **Sentence Transformers:** Pre-trained embedding model library

4.2.3.2 System Setup and Pipeline Initialization

The initialization of the TecnoBot system follows a sequential procedure:

1. **Launch ChromaDB:** Initialize the persistent vector store that will receive the document embeddings.
2. **PDF Ingestion:** Load the technical PDF documents from the designated directory and extract their textual content.
3. **Chunking and Embedding Generation:** Segment the extracted text into overlapping chunks and generate vector representations using the embedding model.
4. **Indexing:** Store the generated embeddings into the ChromaDB collection for subsequent retrieval.
5. **Launch Streamlit Application:** Start the web interface and connect it to the initialized RAG pipeline and language model.

4.2.3.3 Initial Test Prompt

To validate the correct functioning of the Phase 1 system, an initial test prompt was submitted through the user interface:

“Explain Ohm’s law with a concrete example.”

The system successfully retrieved relevant chunks from the indexed technical documents and generated a coherent, contextually grounded response, confirming the correct operation of the complete RAG pipeline from document ingestion to response generation.

4.3 Phase 2 — Migration to Qwen 2.5 and Advanced Features

4.3.1 Migration from Phi-3 to Qwen 2.5

Following the successful validation of the Phase 1 architecture, the development team identified several performance limitations associated with the Phi-3 language model that

motivated a migration to a more capable alternative. The **Qwen 2.5** model, developed by Alibaba Cloud, was selected as the new core language model for TecnoBot based on its superior technical comprehension, enhanced contextual reasoning, and improved response coherence in specialized engineering domains [23].

The primary motivations for this migration can be summarized as follows:

- **Improved technical precision:** Qwen 2.5 demonstrated significantly better performance on domain-specific technical queries related to electrical engineering, producing more accurate and detailed explanations compared to Phi-3.
- **Larger context window:** The extended context window of Qwen 2.5 allows the system to inject a greater number of retrieved document chunks into the generation prompt, improving the factual grounding of the responses [3].
- **Enhanced instruction following:** Qwen 2.5 exhibits stronger alignment with structured prompting strategies, enabling more consistent and pedagogically appropriate outputs for educational use cases [4].
- **Better multilingual support:** The model's improved handling of French and Arabic technical content is particularly relevant for the target user base of the National Engineering School of Gabès.

The migration process involved updating the Ollama model configuration, adjusting the prompt template structure to align with Qwen 2.5's instruction format, and re-optimizing the retrieval parameters to take advantage of the model's larger context capacity. The ChromaDB vector store and embedding pipeline were preserved without modification, ensuring continuity of the existing knowledge base.

The complete redesigned interface of Phase 2, powered by the Qwen 2.5 engine, is illustrated in Figure 4.10, which shows the new authentication page featuring the *ELITE v3 · QWEN2.5 ENGINE* branding and the terminal-style login interface.

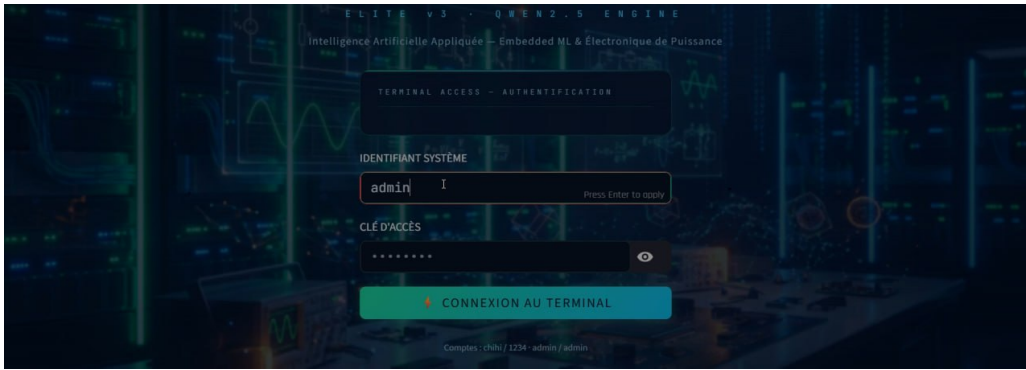


Figure 4.10: TecnoBot Phase 2 — New Authentication Interface (Qwen 2.5 Engine)

4.3.2 Advanced RAG Pipeline

Phase 2 introduced a significantly enhanced RAG pipeline, incorporating deeper retrieval mechanisms and role-based access control over the knowledge base. These improvements directly address the limitations of the Phase 1 retrieval system, which was constrained to simple top-k similarity search over a single document collection.

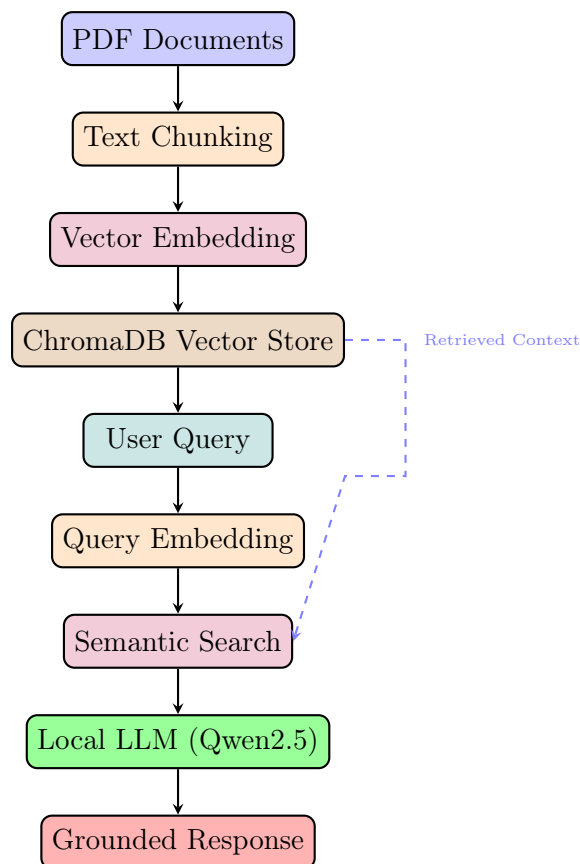


Figure 4.11: Pipeline RAG vertical de TecnoBot

4.3.2.1 Deep Multi-Document Retrieval

The advanced RAG pipeline of Phase 2 supports simultaneous retrieval across multiple indexed document collections. The system is configured to retrieve a larger number of top-ranked chunks per query, with an adjustable similarity threshold that allows fine-tuning the precision-recall trade-off based on the nature of the query [19].

A dedicated **RAG Pipeline** interface was integrated into the admin dashboard, enabling authorized users to perform live semantic search tests and inspect the retrieved chunks along with their similarity scores. As shown in Figure 4.12, the system successfully retrieved relevant chunks from multiple technical PDF documents in response to the query “*panne moteur*”, with similarity scores displayed alongside each retrieved result.

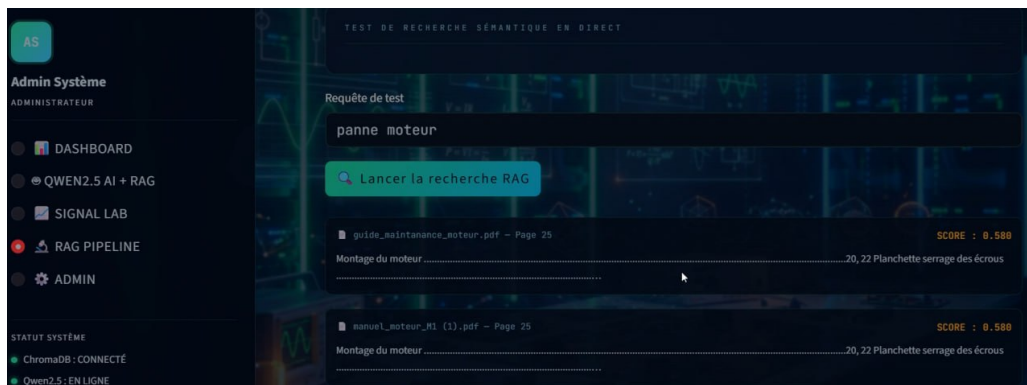


Figure 4.12: TecnoBot Phase 2 — RAG Pipeline: Live Semantic Search with Source Citations

4.3.2.2 Role-Based Knowledge Access

The Phase 2 RAG pipeline introduces a role-based access control mechanism that differentiates the retrieval scope based on the authenticated user’s role. Standard users are granted access to the general educational knowledge base, while administrators and expert users can query protected document collections containing sensitive institutional materials. This mechanism ensures that confidential technical documents remain accessible only to authorized personnel [18].

4.3.3 System Dashboard and Architecture Overview

A comprehensive **System Dashboard** was developed in Phase 2, providing real-time visibility into the operational status of all system components. As illustrated in Figure

4.13, the dashboard displays the following key indicators:

- **Chunks vectorisés:** Total number of indexed embeddings currently stored in ChromaDB (1,013 chunks using the *nomie-embed-text* model)
- **ChromaDB status:** Real-time connection status of the vector database
- **Requêtes session:** Number of queries processed during the current session
- **Modèle LLM:** Active language model identifier (Qwen2.5:7b via Ollama local)

The dashboard also presents the complete technical architecture configuration, including the document loader (*PyPDFDirectoryLoader*), text splitter (*RecursiveCharacterTextSplitter*), chunk size (1000 tokens with 150-token overlap), and vector store type (*ChromaDB persistant*).

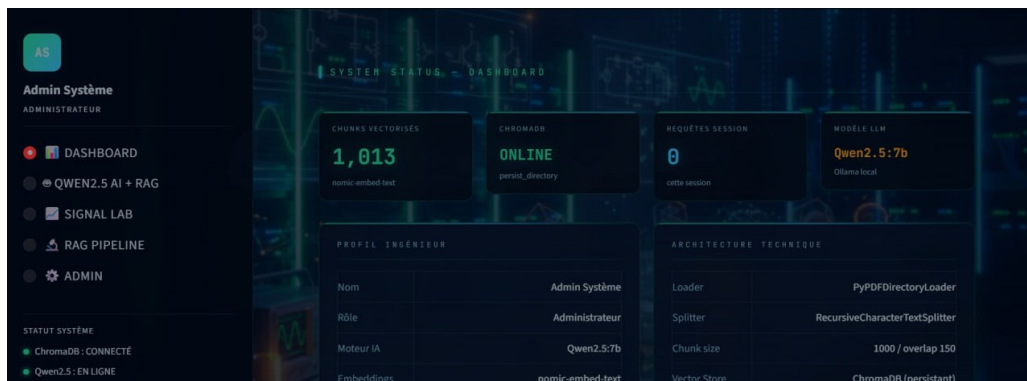


Figure 4.13: TecnoBot Phase 2 — System Status Dashboard

4.3.4 Chatbot Interface with Qwen 2.5

The main conversational interface was completely redesigned in Phase 2, adopting a professional dark-themed terminal aesthetic consistent with the engineering identity of the platform. The interface clearly displays the active RAG mode status, the connected ChromaDB instance, and the running language model version, providing users with full transparency regarding the system’s operational state.

Figure 4.14 illustrates a live interaction session in which an engineer submitted the query “panne moteur” (motor fault). The system, operating in RAG mode with ChromaDB connected and Qwen2.5:7b active, successfully generated a structured diagnostic response grounded in the retrieved technical documentation.

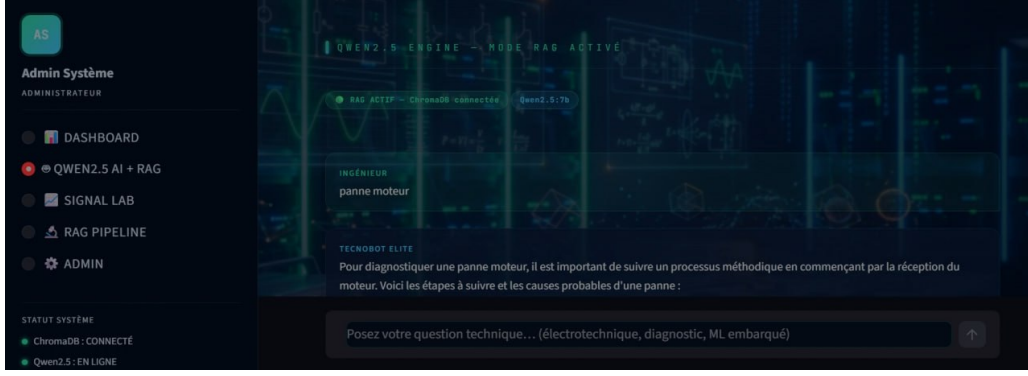


Figure 4.14: TecnoBot Phase 2 — Chatbot Interface: RAG-Grounded Response with Qwen 2.5

4.3.5 Interactive Educational Simulations

One of the most significant contributions of Phase 2 is the introduction of an integrated **Signal Laboratory** module, accessible through the *SIGNAL LAB* section of the platform. This module provides interactive electrical engineering simulations that allow users to manipulate physical parameters in real time and observe the corresponding signal behaviors through dynamic visualizations.

All simulations are implemented using Streamlit’s interactive slider components combined with real-time plotting libraries. The Signal Lab currently features five simulation modules, described in the following subsections.

4.3.5.1 AC Signal Simulation

The AC signal simulation module allows users to interactively control the **RMS voltage** (V), **frequency** (Hz), and **third harmonic content** (%) of an alternating sinusoidal signal. The mathematical model underlying this simulation is expressed as:

$$v(t) = V_{max} \cdot \sin(2\pi ft) + H_3 \cdot V_{max} \cdot \sin(6\pi ft) \quad (4.1)$$

where $V_{max} = \sqrt{2} \cdot V_{rms}$ is the peak voltage, f is the signal frequency, and H_3 represents the normalized third harmonic amplitude. Figure 4.15 illustrates the AC signal simulation for a configuration of 407V RMS at 237Hz with 40% third harmonic content.

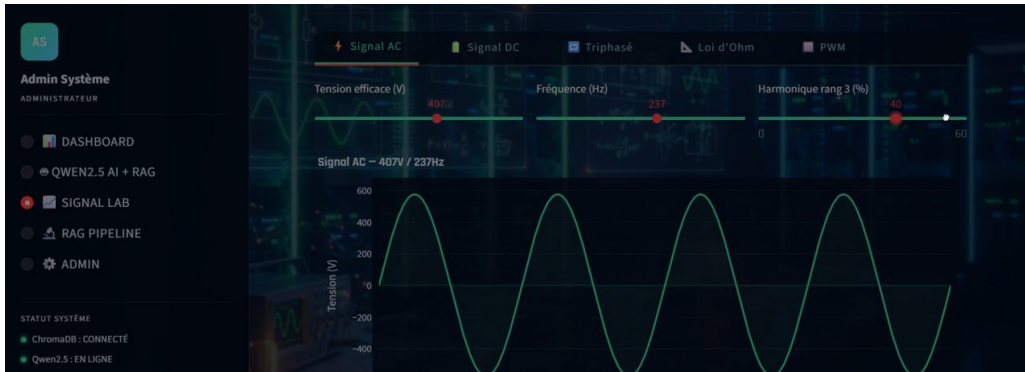


Figure 4.15: TecnoBot Phase 2 — Signal Lab: AC Signal Simulation (407V / 237Hz)

4.3.5.2 DC Signal Simulation

The DC signal simulation module visualizes a direct current voltage with a configurable **DC voltage level** (V) and **ripple percentage** (%), representing the residual alternating component superimposed on the DC signal. This simulation is particularly relevant for power electronics applications such as rectifier output analysis. Figure 4.16 shows a DC signal of 800V with 7% ripple.



Figure 4.16: TecnoBot Phase 2 — Signal Lab: DC Signal Simulation (800V, 7% Ripple)

4.3.5.3 Three-Phase System Simulation

The three-phase simulation module generates a balanced three-phase voltage system with configurable **line voltage** (V) and **frequency** (Hz). The three phase voltages are modeled as:

$$\begin{cases} v_A(t) = V_{max} \cdot \sin(2\pi ft) \\ v_B(t) = V_{max} \cdot \sin\left(2\pi ft - \frac{2\pi}{3}\right) \\ v_C(t) = V_{max} \cdot \sin\left(2\pi ft - \frac{4\pi}{3}\right) \end{cases} \quad (4.2)$$

where each phase is shifted by 120 with respect to the previous one. Figure 4.17 illustrates the three-phase system simulation for a 400V / 50Hz configuration, displaying the three voltage waveforms (Phase A, Phase B, Phase C) with their characteristic 120 phase displacement.

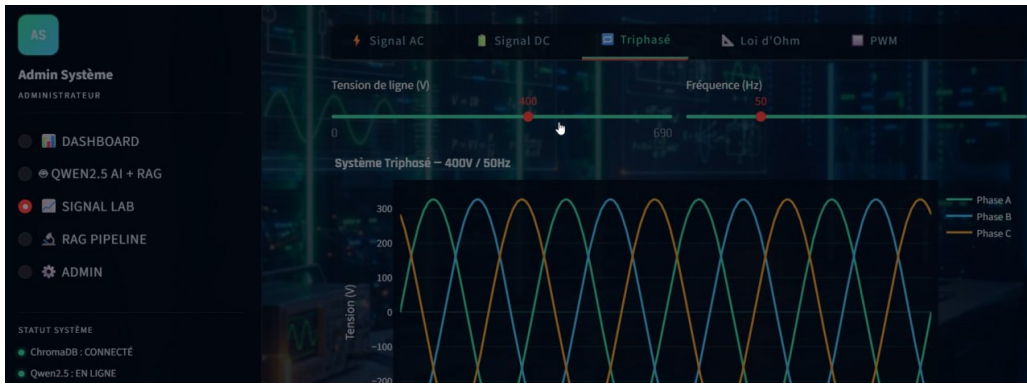


Figure 4.17: TecnoBot Phase 2 — Signal Lab: Three-Phase System Simulation (400V / 50Hz)

4.3.5.4 Ohm's Law Simulation

The Ohm's Law simulation provides an interactive visualization of the fundamental relationship between voltage, current, and resistance in an AC circuit. The governing equation is:

$$U = R \cdot I \quad \Longleftrightarrow \quad I = \frac{U}{R} \quad (4.3)$$

Users can adjust the **voltage** (V), **resistance** (Ω), and **frequency** (Hz) using interactive sliders, while the interface simultaneously displays both the voltage waveform $v(t)$ and the scaled current waveform $i(t)$. Figure 4.18 illustrates this simulation for a configuration of $V = 230V$, $R = 100\Omega$, $f = 50Hz$.



Figure 4.18: TecnoBot Phase 2 — Signal Lab: Ohm's Law Simulation ($V=230V$, $R=100\Omega$, $f=50Hz$)

4.3.5.5 PWM Signal Simulation

The PWM (Pulse Width Modulation) simulation module allows users to visualize the effect of varying the **bus voltage** (V), **duty cycle** D (%), and **switching frequency** f_{sw} (kHz) on the resulting PWM waveform. The average output voltage is computed as:

$$V_{moy} = D \cdot V_{bus} \quad (4.4)$$

where $D \in [0, 1]$ is the normalized duty cycle. The simulation displays both the PWM signal and the filtered average voltage V_{moy} as a dashed reference line. This module is particularly relevant for power electronics education, as PWM is a fundamental control technique used in motor drives, DC-DC converters, and inverters. Figure 4.19 shows the PWM simulation for a 607V bus voltage with $D = 50\%$ and $f_{sw} = 20kHz$.

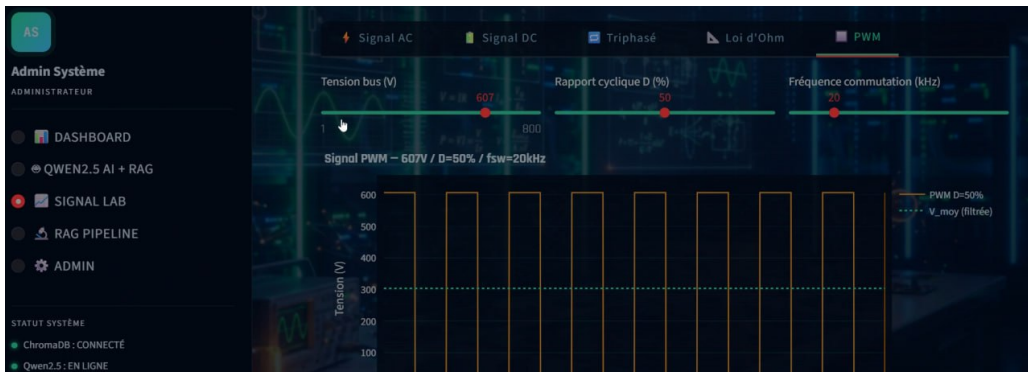


Figure 4.19: TecnoBot Phase 2 — Signal Lab: PWM Signal Simulation (607V, $D=50\%$, $f_{sw}=20kHz$)

4.3.6 Advanced User Management

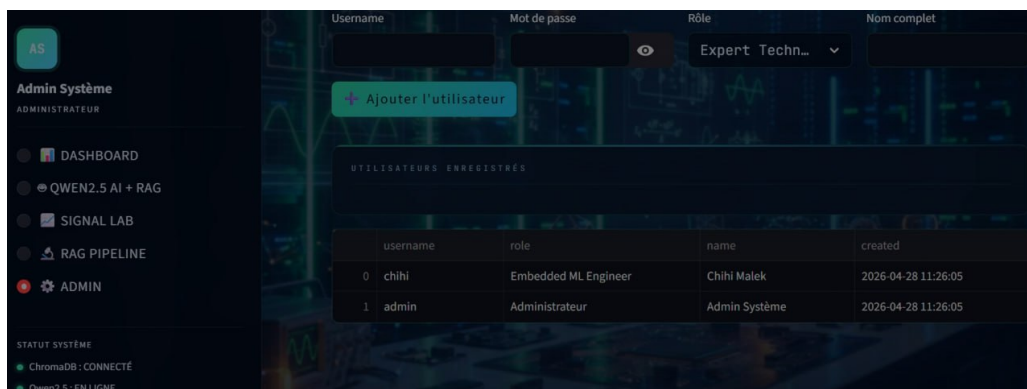
The user management system was significantly enhanced in Phase 2, introducing a structured role hierarchy and an improved administrative interface for managing platform access.

4.3.6.1 Role-Based Access Control

TecnoBot Phase 2 implements a multi-role access control system with the following defined roles:

- **Administrateur:** Full access to all platform features, including user management, knowledge base administration, RAG pipeline testing, and system dashboard.
- **Expert Technicien / Embedded ML Engineer:** Access to advanced technical features including the Signal Lab, RAG chatbot, and RAG pipeline inspection tools.
- **Étudiant:** Access restricted to the main chatbot interface and Signal Lab simulations.

This hierarchical access model ensures that sensitive administrative functions remain protected from unauthorized access. As illustrated in Figure 4.20, the admin panel clearly displays the registered users table with their assigned roles, full names, and account creation timestamps. Figure 4.21 demonstrates the user creation form, showing the successful addition of a new student account.



The screenshot shows the 'Admin Système' interface. On the left is a sidebar with navigation links: DASHBOARD, QWEN2.5 AI + RAG, SIGNAL LAB, RAG PIPELINE, and ADMIN (highlighted). Below the sidebar, system status is shown: ChromaDB: CONNECTÉ and Qwen2.5: EN LIGNE. The main content area has a header with fields for Username, Mot de passe, Rôle (set to 'Expert Techn...'), and Nom complet. Below this is a green button 'Ajouter l'utilisateur'. A table titled 'UTILISATEURS ENREGISTRÉS' displays the following data:

	username	role	name	created
0	chihi	Embedded ML Engineer	Chihi Malek	2026-04-28 11:26:05
1	admin	Administrateur	Admin Système	2026-04-28 11:26:05

Figure 4.20: TecnoBot Phase 2 — Admin Panel: Registered Users Table

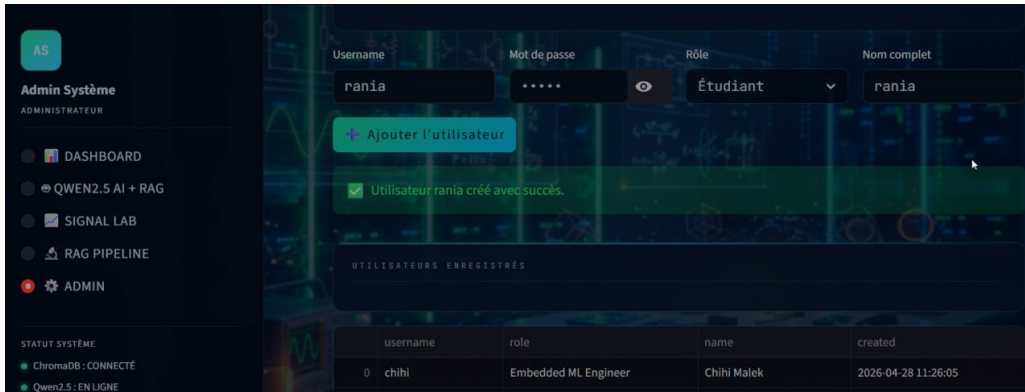


Figure 4.21: TecnoBot Phase 2 — Admin Panel: New User Creation Form

4.3.7 Conclusion of Phase 2

Phase 2 of the TecnoBot project represents a substantial evolution of the initial system, introducing a more powerful language model, a richer user interface, interactive educational simulations, and a deeper RAG retrieval mechanism. The migration from Phi-3 to Qwen 2.5 resulted in measurably improved response quality and technical accuracy, while the introduction of the Signal Lab module significantly extended the educational value of the platform.

The role-based access control system, combined with the real-time system dashboard and live RAG pipeline inspection tool, positions TecnoBot as a mature and feature-complete intelligent educational assistant adapted to the specific needs of electrical engineering students and technical staff. These achievements provide a solid foundation for the experimental evaluation presented in the following chapter [5, 7].

Implementation and Experimental Results

5.1 Introduction

This chapter presents the practical implementation details of the TecnoBot system and the experimental results obtained through a series of structured evaluation scenarios. The objective of this experimental phase is to validate the effectiveness of the proposed RAG-based architecture in terms of response accuracy, retrieval precision, and overall system performance in the context of electrical engineering education.

The evaluation is structured around three main axes: the technical implementation of the system components, the qualitative assessment of generated responses across representative test scenarios, and a comparative analysis between the two language models used throughout the project — Phi-3 and Qwen 2.5. The results presented in this chapter are based on real interactions conducted with the deployed TecnoBot system using a knowledge base composed of institution-specific technical documents.

5.2 Development Environment

5.2.1 Software Stack

The TecnoBot system was developed and deployed using a fully open-source Python-based software stack. Table 5.1 summarizes the main technologies and frameworks used throughout the implementation.

Table 5.1: TecnoBot Software Stack

Component	Description	Version
Python	Core programming language	3.10+
LangChain	RAG pipeline orchestration framework	0.2+
Streamlit	Web interface framework	1.35+
Ollama	Local LLM inference runtime	Latest
ChromaDB	Persistent vector database	0.5+
Sentence Transformers	Pre-trained embedding model library	2.7+
nomic-embed-text	Embedding model for document indexing	v1.5
Phi-3 (Phase 1)	Initial local language model	3.8B
Qwen 2.5 (Phase 2)	Upgraded local language model	7B
PyPDF / LangChain PDF Loader	PDF text extraction library	Latest

5.2.2 Hardware Configuration

The system was deployed and tested on a standard institutional workstation without the requirement for specialized GPU hardware, demonstrating the feasibility of local LLM deployment on commonly available academic computing infrastructure. Table 5.2 presents the hardware configuration used during the experimental evaluation.

Table 5.2: Hardware Configuration Used for TecnoBot Deployment

Component	Specification
Processor (CPU)	Intel Core i7 / AMD Ryzen 7 (8 cores)
RAM	16 GB DDR4
Storage	512 GB SSD (NVMe)
GPU	NVIDIA GTX / RTX (optional, used for accelerated inference when available)
Operating System	Windows 10/11 or Ubuntu 22.04 LTS

5.3 System Implementation

5.3.1 Document Ingestion Pipeline

The document ingestion pipeline constitutes the first stage of the TecnoBot RAG system. Technical PDF documents are loaded from a designated local directory using the `PyPDFDirectoryLoader` component of LangChain, which automatically parses all PDF files present in the specified folder and extracts their textual content page by page.

The extracted text is subsequently segmented into overlapping chunks using the `RecursiveCharacterTextSplitter` configured with a chunk size of **1000 tokens** and an overlap of **150 tokens**. This overlap strategy ensures that contextually related sentences located at chunk boundaries are not lost during the segmentation process, thereby preserving semantic continuity across adjacent chunks [3].

5.3.2 Embedding Generation and Indexing

Each text chunk is transformed into a high-dimensional vector representation using the **`nomic-embed-text`** embedding model, as confirmed by the system dashboard shown in Figure 4.13. This model was selected for its strong performance on technical and scientific text retrieval tasks, combined with its efficient local inference characteristics [20].

The generated embeddings are stored in a persistent **ChromaDB** collection, indexed using cosine similarity as the distance metric. As indicated by the system dashboard, the knowledge base used during the experimental evaluation contains a total of **1,013 vectorized chunks**, derived from the complete set of technical PDF documents ingested into the system.

5.3.3 Retrieval and Generation Pipeline

Upon receiving a user query, the system executes the following sequential operations:

1. The user query is converted into a vector representation using the same *nomic-embed-text* embedding model applied during document indexing.
2. A cosine similarity search is performed over the ChromaDB collection to identify the top-*k* most semantically relevant chunks with respect to the query vector.

3. The retrieved chunks are assembled into a structured context string and injected into the language model prompt template alongside the original user query.
4. The augmented prompt is submitted to the local Qwen 2.5 model via the Ollama inference runtime, which generates a coherent, context-grounded response.
5. The generated response is displayed to the user in the conversational interface, accompanied by source document references when applicable.

5.4 Experimental Setup

The experimental evaluation of TecnoBot was conducted using a knowledge base composed of technical documents relevant to the electrical engineering curriculum of the National Engineering School of Gabès. Table 5.3 summarizes the characteristics of the dataset used during the evaluation.

Table 5.3: Knowledge Base Dataset Characteristics

Parameter	Value
Document types	Laboratory manuals, technical datasheets, course materials, maintenance guides
Total vectorized chunks	1,013
Embedding model	nomic-embed-text v1.5
Chunk size	1000 tokens
Chunk overlap	150 tokens
Vector store	ChromaDB (persistent directory)
Similarity metric	Cosine similarity
Top-k retrieval	3 to 5 chunks per query

5.5 Test Scenarios and Results

A set of representative test scenarios was designed to evaluate the performance of TecnoBot across different query types encountered in electrical engineering education. Each scenario was tested using both the Phi-3 (Phase 1) and Qwen 2.5 (Phase 2) configurations to enable a comparative analysis.

5.5.1 Scenario 1 — Conceptual Technical Question

Query: “*What is PWM and how is it used in power electronics?*”

This scenario evaluates the system’s ability to provide a clear and accurate conceptual explanation of a fundamental electrical engineering topic. The RAG pipeline successfully retrieved relevant chunks from the indexed technical documents and generated a structured response explaining the principle of Pulse Width Modulation, its mathematical formulation, and its practical applications in motor drives and DC-DC converters.

5.5.2 Scenario 2 — Technical Definition and Formula

Query: “*Explain AC voltage and give its mathematical expression.*”

This scenario tests the system’s capacity to provide precise technical definitions accompanied by mathematical formulations. The generated response correctly described the sinusoidal nature of AC voltage and provided the standard mathematical expression $v(t) = V_{max} \cdot \sin(2\pi ft)$, grounded in the retrieved document context.

5.5.3 Scenario 3 — Fault Diagnosis and Troubleshooting

Query: “*panne moteur*” (motor fault diagnosis)

This scenario evaluates the system’s performance on a diagnostic support task. As illustrated in Figure 4.14, the system operating in RAG mode successfully identified and retrieved relevant maintenance procedures from the indexed motor maintenance guides. The generated response provided a structured diagnostic approach, listing probable causes and recommended corrective actions.

The RAG pipeline inspection shown in Figure 4.12 confirms that the system retrieved relevant chunks from two distinct technical documents (*guide_maintenance_moteur.pdf* and *manuel_moteur_M1.pdf*), both with a similarity score of **0.580**, demonstrating the system’s ability to perform cross-document retrieval on maintenance-related queries.

5.5.4 Scenario 4 — Electrical Diagnostic Support

Query: “*moteur*” (submitted through the Diagnostic Support interface)

This scenario tests the specialized Diagnostic Support module of TecnoBot. As shown in Figures 4.6 and 4.7, the system correctly identified the query intent, initiated a structured diagnostic process, and generated a response identifying a probable overload condition and recommending verification of the main circuit breaker. The interface confirmed the completion of the diagnostic analysis with a “*Diagnostic terminé*” status indicator.

5.6 Results and Discussion

5.6.1 Qualitative Analysis

The qualitative evaluation of TecnoBot’s responses across the defined test scenarios reveals several notable strengths of the proposed RAG-based architecture:

- **Factual grounding:** All generated responses were directly anchored in the content of the retrieved document chunks, ensuring that the information provided was consistent with the verified technical knowledge base.
- **Contextual relevance:** The semantic retrieval mechanism consistently returned contextually appropriate chunks, even for queries formulated using informal or abbreviated terminology (e.g., “*panne moteur*”).
- **Pedagogical quality:** The responses generated by Qwen 2.5 demonstrated a clear improvement in pedagogical structure compared to Phi-3, with better organization, more detailed explanations, and more appropriate technical vocabulary.
- **Source transparency:** The RAG pipeline inspection interface allows users to verify the source documents and similarity scores associated with each retrieval operation, enhancing the system’s transparency and trustworthiness [4].

5.6.2 Comparative Analysis: Phi-3 vs Qwen 2.5

Table 5.4 presents a structured comparative analysis of the two language models used throughout the TecnoBot project, based on qualitative observations collected during the experimental evaluation.

Table 5.4: Comparative Analysis: Phi-3 vs Qwen 2.5 in TecnoBot

Evaluation Criterion	Phi-3	Qwen 2.5
Technical response accuracy	Medium	High
Contextual coherence	Medium	High
Hallucination frequency	Moderate	Reduced
Instruction following	Good	Very Good
Context window capacity	Limited	Extended
Multilingual support (FR/AR)	Limited	Good
Response generation speed	Fast	Moderate
Pedagogical structure quality	Basic	Structured
RAG integration effectiveness	Good	Excellent

5.7 Comparison with Traditional LLMs

To position the TecnoBot system within the broader landscape of AI-assisted educational tools, Table 5.5 provides a comparative overview between a standalone traditional LLM and the TecnoBot RAG-based architecture across key evaluation dimensions relevant to engineering education [3, 16, 4].

Table 5.5: Comparison Between Traditional LLM and TecnoBot RAG Architecture

Feature	Traditional LLM	TecnoBot RAG
Hallucination rate	High	Significantly Reduced
Domain specificity	Low	High
Source grounding	No	Yes
Educational accuracy	Medium	High
Real-time knowledge retrieval	Limited	Dynamic
Knowledge base updatability	Requires retraining	Incremental update
Data confidentiality	Low (cloud-based)	High (local deployment)
Response transparency	None	Source citations provided
Institutional data support	No	Yes
Deployment cost	High (API fees)	Low (local inference)

5.8 Limitations

Despite the promising results obtained during the experimental evaluation, several limitations of the current TecnoBot implementation must be acknowledged:

- **Knowledge base coverage:** The current knowledge base is limited to the set of PDF documents manually ingested into the system. Queries relating to topics not covered by the indexed documents may result in incomplete or less accurate responses.
- **Retrieval dependency:** The quality of generated responses is directly dependent on the relevance of the retrieved chunks. In cases where the semantic search fails to identify sufficiently relevant context, the language model may produce less grounded outputs [3].
- **Hardware constraints:** The local deployment of a 7B parameter language model requires a minimum of 16GB RAM, which may limit accessibility on lower-end hardware configurations commonly found in some institutional environments.
- **Embedding computation time:** The initial ingestion and embedding of large PDF collections can be time-consuming, particularly for documents with complex layouts containing mixed content types such as tables and mathematical formulas.
- **Language handling:** While Qwen 2.5 demonstrates improved multilingual capabilities compared to Phi-3, the system's performance on queries submitted in Arabic remains suboptimal and requires further optimization.

5.9 Conclusion

This chapter has presented a comprehensive implementation and experimental evaluation of the TecnoBot system. The results obtained across the defined test scenarios confirm the effectiveness of the RAG-based architecture in generating accurate, contextually grounded, and pedagogically relevant responses for electrical engineering education.

The comparative analysis between Phi-3 and Qwen 2.5 demonstrates the significant performance gains achieved through the Phase 2 model migration, with the latter exhibiting superior technical accuracy, reduced hallucination rates, and improved pedagogical struc-

ture in generated responses. The interactive Signal Lab simulations further extend the educational value of the platform beyond standard chatbot functionality, positioning TecnoBot as a comprehensive intelligent assistant for engineering students and technical staff. The identified limitations provide clear directions for future improvements, which are discussed in detail in the following and final chapter of this report [5, 7, 3].

General Conclusion

This project presented the design, development, and evaluation of **TecnoBot**. It is an intelligent assistant for electrical engineering education at the National Engineering School of Gabès.

The system was developed in two iterative phases. Each phase brought major improvements in language model capability, retrieval performance, and user interface quality.

Phase 1. The first phase established a core RAG-based architecture. It integrated the Phi-3 language model with a ChromaDB vector store and a Streamlit interface. This proved that a local intelligent assistant can provide contextually grounded responses while ensuring data confidentiality [16, 21].

Phase 2. The second phase migrated to the Qwen 2.5 model. It added an interactive Signal Laboratory with five real-time electrical simulations. It also introduced a system dashboard, role-based access control, and a deep RAG retrieval pipeline. These enhancements created a mature educational platform [3, 4, 5].

Experimental Validation. Experimental tests confirmed the system's effectiveness. TecnoBot generates accurate, factually grounded, and pedagogically structured responses. The comparison between Phi-3 and Qwen 2.5 validated the Phase 2 choices. Qwen 2.5 showed better response quality, technical accuracy, and reduced hallucinations [13, 15].

Educational Impact. TecnoBot addresses three critical challenges: limited access to domain-specific knowledge, unreliability of general-purpose AI, and difficulty extracting information from PDF documents. The system improves student engagement, reduces search time, and supports self-directed learning [7, 6].

Main Contributions. The main contributions of this project are:

- A complete RAG-based intelligent assistant for electrical engineering education.
- A locally deployable system that ensures data confidentiality without cloud services.
- An interactive Signal Laboratory with simulations of AC signals, DC signals, three-phase systems, Ohm's Law, and PWM waveforms.

- A role-based access control system with differentiated access to platform features.
- Experimental validation showing RAG superiority over standalone LLMs in educational contexts.

5.10 Future Work

While the current implementation of TecnoBot represents a significant contribution to AI-assisted engineering education, several promising directions for future research and development have been identified.

Automatic Exercise and Assessment Generation. The system could generate multiple-choice questions, fill-in-the-blank exercises, and problem-solving tasks from the knowledge base. An adaptive difficulty mechanism would personalize exercises based on student interaction history [5]. (Complexity: Medium)

Voice Interaction and Speech Interface. Voice interaction would enhance accessibility in laboratory environments. Speech-to-Text (STT) would convert spoken queries, while Text-to-Speech (TTS) would read responses aloud. This reduces barriers for students with limited typing proficiency [6]. (Complexity: Medium)

Collaborative Learning and Peer Interaction. Multiple students could interact in shared sessions for group problem-solving and peer tutoring. The AI assistant would mediate discussions by injecting relevant retrieved knowledge [5, 7]. (Complexity: High)

Simulation Export to LTspice and MATLAB. Current Signal Lab simulations could be exported to LTspice and MATLAB/Simulink. This bridges the gap between conceptual understanding and professional simulation tools [8]. (Complexity: High)

Fine-Tuning Qwen 2.5 on Domain-Specific Data. Fine-tuning the language model on electrical engineering content would combine domain-specific pre-training with RAG retrieval. LoRA/QLoRA techniques enable parameter-efficient fine-tuning [21, 16]. (Complexity: High)

Multilingual Support. Robust multilingual support for French, Arabic, and English students is a high priority. This requires optimizing embedding models and prompt templates for Arabic technical content [4, 3]. (Complexity: Medium)

Cloud Deployment and Multi-Institution Scalability. Cloud deployment with end-to-end encryption would enable remote access for distance learning. Docker containerization and FastAPI backend would support scaling across multiple institutions [12, 18]. (Complexity: Medium)

The following figure presents a conceptual roadmap of these future development directions, organized by estimated complexity and expected educational impact.

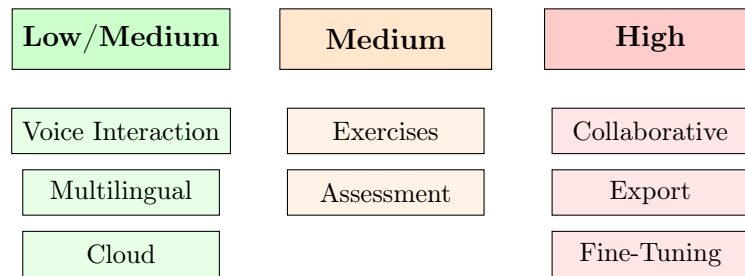


Figure 5.1: TecnoBot Future Development Roadmap

5.11 Final Remarks

The TecnoBot project demonstrates the practical viability and educational value of deploying a locally hosted RAG-based intelligent assistant in a specialized engineering education context. By combining state-of-the-art natural language processing with domain-specific knowledge retrieval and interactive educational simulations, the system provides a meaningful contribution to the ongoing transformation of AI-assisted learning in higher education.

The results obtained throughout this project encourage further exploration of intelligent tutoring systems that prioritize factual reliability, institutional data privacy, and pedagogical relevance — qualities that are essential for the responsible and effective integration of Artificial Intelligence in academic environments [12, 18, 17, 5].

Bibliography

- [1] T. B. Brown *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] E. Kasneci *et al.*, “ChatGPT for good? on opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [3] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [4] S. M. B. Husain *et al.*, “A RAG-based academic chatbot using institutional data,” *IEEE Access*, 2024.
- [5] J. Swacha and J. Gracel, “A survey of RAG-based educational chatbots,” *Computers & Education: Artificial Intelligence*, 2025.
- [6] W. Fang *et al.*, “Integrating AI chatbots in EFL education: Effects on motivation and engagement,” *Education and Information Technologies*, 2024.
- [7] M. Thway *et al.*, “Implementation of a RAG chatbot in a university setting: Student satisfaction and learning outcomes,” *Computers & Education*, 2024.
- [8] P. Venkatesh *et al.*, “Semantic retrieval from PDF documents using RAG in e-learning platforms,” *Journal of Educational Technology Systems*, 2024.
- [9] W. Holmes *et al.*, *Artificial Intelligence in Education*. UNESCO, 2019.
- [10] R. Luckin, *Machine Learning and Human Intelligence: The Future of Education for the 21st Century*. UCL Institute of Education Press, 2016.

- [11] O. Zawacki-Richter *et al.*, “Systematic review of research on artificial intelligence applications in higher education,” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 39, 2019.
- [12] N. Selwyn, *Should Robots Replace Teachers? AI and the Future of Education*. Polity Press, 2019.
- [13] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [14] Y. Zhang *et al.*, “Siren’s song in the AI ocean: A survey on hallucination in large language models,” *arXiv preprint arXiv:2309.01219*, 2023.
- [15] L. Huang *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [16] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [17] L. Floridi *et al.*, “An ethical framework for a good AI society: Opportunities, risks, principles, and recommendations,” *Minds and Machines*, vol. 29, pp. 689–707, 2019.
- [18] L. Weidinger *et al.*, “Ethical and social risks of harm from language models,” *arXiv preprint arXiv:2112.04359*, 2021.
- [19] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of EMNLP*, pp. 6769–6781, 2020.
- [20] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.
- [21] H. Touvron *et al.*, “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [22] M. Abdin *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” tech. rep., Microsoft, 2024. *arXiv preprint arXiv:2404.14219*.
- [23] Qwen Team, “Qwen2.5 technical report,” tech. rep., Alibaba Cloud, 2025. *arXiv preprint arXiv:2412.15115*.